RoboPet — Phase 3: Combat

Auto-Battler Simulation, resolveStats & Ghost PvP

Goal. Extend the room-based architecture with a fourth room -- the Combat Arena. resolveStats computes HP/ATK/DEF from equipped modules. CombatNotifier drives a turn-based auto-battler. CombatRoomWorld renders the fight with shapes + particles. The result is written to Firestore as a CombatSnapshot for async ghost PvP. Zero breaking changes to existing providers, Isar schema, or room switching.

1. Domain: resolveStats

lib/domain/combat/resolved_stats.dart

```
/// Final combat stats after module modifiers are applied.
class ResolvedCombatStats {
  const ResolvedCombatStats({
    required this.hp,
    required this.attack,
    required this.defense,
    required this.speed,
    required this.energyRegen,
  });

  final int hp;
  final int attack;
  final int defense;
  final int speed;
  final int energyRegen;

  /// Dummy ghost opponent with slightly boosted base stats.
  static const ghost = ResolvedCombatStats(
    hp: 120, attack: 18, defense: 12, speed: 10, energyRegen: 2);

  @override
  String toString() => 'HP:$hp ATK:$attack DEF:$defense SPD:$speed';
}
```

lib/domain/combat/stat_resolver.dart

```
import '../../data/models/module_instance.dart';
import '../../data/models/module_enums.dart';
import 'resolved_stats.dart';

/// Base stats every robot starts with before module bonuses.
const _kBase = (
  hp: 100.0, attack: 15.0, defense: 10.0, speed: 10.0, energyRegen: 2.0);

/// Formula: (base + sumFlat) * (1 + sumPct) -- per stat, per module.
ResolvedCombatStats resolveStats(List<ModuleInstance> equippedModules) {
  final flat = <StatKey, double>{};
  final pct = <StatKey, double>{};

  for (final m in equippedModules) {
    final def = m.def;
```

```
    if (def == null) continue;
    for (final mod in def.modifiers) {
      flat[mod.stat] = (flat[mod.stat] ?? 0) + mod.flat;
      pct[mod.stat] = (pct[mod.stat] ?? 0) + mod.pct;
    }
  }

  int r(StatKey key, double base) =>
      ((base + (flat[key] ?? 0)) * (1 + (pct[key] ?? 0))).round();

  return ResolvedCombatStats(
    hp: r(StatKey.hp, _kBase.hp),
    attack: r(StatKey.attack, _kBase.attack),
    defense: r(StatKey.defense, _kBase.defense),
    speed: r(StatKey.speed, _kBase.speed),
    energyRegen: r(StatKey.energyRegen, _kBase.energyRegen),
  );
}
```

---

2. CombatSnapshot (Isar + Firestore)

lib/data/models/combat_snapshot.dart

```
import 'package:isar/isar.dart';

part 'combat_snapshot.g.dart';

@collection
class CombatSnapshot {
  Id id = Isar.autoIncrement;

  @Index(unique: true, replace: true)
  late String snapshotId;

  late String ownerUid;
  late String robotInstanceId;

  int resolvedHp = 100;
  int resolvedAttack = 15;
  int resolvedDefense = 10;
  int resolvedSpeed = 10;

  List<String> equippedDefIds = [];

  late DateTime createdAt;
  int schemaVersion = 3;
}
```

Re-run codegen: dart run build_runner build --delete-conflicting-outputs

---

3. CombatSyncService (push snapshot to Firestore)

lib/data/sync/combat_sync_service.dart

```
import 'package:cloud_firestore/cloud_firestore.dart';
```

```dart
import 'package:isar/isar.dart';
import 'package:uuid/uuid.dart';

import '../models/combat_snapshot.dart';
import '../models/module_instance.dart';
import '../../domain/combat/resolved_stats.dart';

class CombatSyncService {
  CombatSyncService({required this.isar, required this.db, required this.uid});

  final Isar isar;
  final FirebaseFirestore db;
  final String uid;

  Future<CombatSnapshot> push({
    required String robotInstanceId,
    required ResolvedCombatStats stats,
    required List<ModuleInstance> equipped,
  }) async {
    final snap = CombatSnapshot()
      ..snapshotId = const Uuid().v4()
      ..ownerUid = uid
      ..robotInstanceId = robotInstanceId
      ..resolvedHp = stats.hp
      ..resolvedAttack = stats.attack
      ..resolvedDefense = stats.defense
      ..resolvedSpeed = stats.speed
      ..equippedDefIds = equipped.map((m) => m.defId).toList()
      ..createdAt = DateTime.now().toUtc()
      ..schemaVersion = 3;

    await isar.writeTxn(() => isar.combatSnapshots.put(snap));

    await db.doc('users/$uid/combatSnapshot/active').set({
      'snapshotId': snap.snapshotId,
      'resolvedHp': snap.resolvedHp,
      'resolvedAttack': snap.resolvedAttack,
      'resolvedDefense': snap.resolvedDefense,
      'resolvedSpeed': snap.resolvedSpeed,
      'equippedDefIds': snap.equippedDefIds,
      'robotInstanceId': snap.robotInstanceId,
      'createdAt': FieldValue.serverTimestamp(),
    });

    return snap;
  }
}
```

```
flutter pub add uuid
```

Firestore rule to add: match /users/{uid}/combatSnapshot/{doc} { allow read; allow write: if request.auth.uid == uid; }

---

4. CombatNotifier (pure Dart, Riverpod)

`lib/data/combat/combat_notifier.dart`

```dart
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../../domain/combat/resolved_stats.dart';
```

```
enum CombatPhase { idle, battling, victory, defeat }

class CombatState {
  const CombatState({
    this.phase = CombatPhase.idle,
    this.playerHp = 0, this.playerMaxHp = 0,
    this.enemyHp = 0, this.enemyMaxHp = 0,
    this.playerAtk = 0, this.playerDef = 0,
    this.enemyAtk = 0, this.enemyDef = 0,
    this.log = const [], this.turnCount = 0,
  });

  final CombatPhase phase;
  final int playerHp, playerMaxHp, enemyHp, enemyMaxHp;
  final int playerAtk, playerDef, enemyAtk, enemyDef;
  final List<String> log;
  final int turnCount;

  double get playerHpFraction =>
      playerMaxHp == 0 ? 0 : (playerHp / playerMaxHp).clamp(0.0, 1.0);
  double get enemyHpFraction =>
      enemyMaxHp == 0 ? 0 : (enemyHp / enemyMaxHp).clamp(0.0, 1.0);

  CombatState copyWith({
    CombatPhase? phase, int? playerHp, int? playerMaxHp,
    int? enemyHp, int? enemyMaxHp, int? playerAtk, int? playerDef,
    int? enemyAtk, int? enemyDef, List<String>? log, int? turnCount,
  }) => CombatState(
    phase: phase ?? this.phase,
    playerHp: playerHp ?? this.playerHp,
    playerMaxHp: playerMaxHp ?? this.playerMaxHp,
    enemyHp: enemyHp ?? this.enemyHp,
    enemyMaxHp: enemyMaxHp ?? this.enemyMaxHp,
    playerAtk: playerAtk ?? this.playerAtk,
    playerDef: playerDef ?? this.playerDef,
    enemyAtk: enemyAtk ?? this.enemyAtk,
    enemyDef: enemyDef ?? this.enemyDef,
    log: log ?? this.log,
    turnCount: turnCount ?? this.turnCount,
  );
}

class CombatNotifier extends StateNotifier<CombatState> {
  CombatNotifier() : super(const CombatState());

  void startBattle({required ResolvedCombatStats player,
      required ResolvedCombatStats enemy}) {
    state = CombatState(
      phase: CombatPhase.battling,
      playerHp: player.hp, playerMaxHp: player.hp,
      enemyHp: enemy.hp, enemyMaxHp: enemy.hp,
      playerAtk: player.attack, playerDef: player.defense,
      enemyAtk: enemy.attack, enemyDef: enemy.defense,
      log: ['Battle started!']);
  }

  bool tick() {
    if (state.phase != CombatPhase.battling) return false;
    final log = List<String>.from(state.log);

    final pDmg = _damage(state.playerAtk, state.enemyDef);
    var eHp = (state.enemyHp - pDmg).clamp(0, state.enemyMaxHp);
    log.add('Your robot deals $pDmg DMG. Enemy HP: $eHp');

    if (eHp <= 0) {
      state = state.copyWith(phase: CombatPhase.victory, enemyHp: 0,
          log: log..add('Victory! Enemy defeated.'));
      return true;
    }
```

```
    final eDmg = _damage(state.enemyAtk, state.playerDef);
    var pHp = (state.playerHp - eDmg).clamp(0, state.playerMaxHp);
    log.add('Enemy deals $eDmg DMG. Your HP: $pHp');

    if (pHp <= 0) {
      state = state.copyWith(phase: CombatPhase.defeat, playerHp: 0,
          log: log..add('Defeated! Robot went offline.'));
      return true;
    }

    state = state.copyWith(
      playerHp: pHp, enemyHp: eHp,
      log: log.length > 12 ? log.sublist(log.length - 12) : log,
      turnCount: state.turnCount + 1);
    return true;
  }

  void reset() => state = const CombatState();

  static int _damage(int atk, int def) =>
      (atk - def * 0.3).round().clamp(1, 9999);
}
```

---

5. Riverpod providers (additions to providers.dart)

```
import 'data/combat/combat_notifier.dart';
import 'data/sync/combat_sync_service.dart';
import 'domain/combat/stat_resolver.dart';
import 'domain/combat/resolved_stats.dart';

// Combat state.
final combatProvider =
    StateNotifierProvider<CombatNotifier, CombatState>(
        (ref) => CombatNotifier());

// Resolved stats for the active robot.
final resolvedStatsProvider = Provider<ResolvedCombatStats>((ref) {
  const robotId = 'robot_0'; // swap with ref.watch(activeRobotIdProvider)
  final equipped = ref.watch(equippedModulesProvider(robotId));
  return resolveStats(equipped.values.toList());
});

// CombatSyncService.
final combatSyncProvider = Provider<CombatSyncService>((ref) {
  final user = ref.watch(sessionProvider).requireValue;
  return CombatSyncService(
    isar: ref.watch(isarProvider),
    db: ref.watch(firestoreProvider),
    uid: user.uid,
  );
});
```

---

6. Extend RoboPetGame (minimal diff)

Only four additions. Nothing is removed or modified. Existing room switching works unchanged.

```
// 1) Add combat to the Room enum:
enum Room { living, fabricator, laboratory, combat }

// 2) New overlay key constant:
const kOverlayCombat = 'combat';

// 3) Inside RoboPetGame class:
import 'rooms/combat_room.dart';

late final CombatRoomWorld _combat;
CombatRoomWorld get combatWorld => _combat; // public getter

// In onLoad(), after existing room inits:
_combat = CombatRoomWorld();

// 4) Extend the two switch expressions:
// _toggleRoomOverlay:
Room.combat => kOverlayCombat,
Room.living => null,       // existing line

// _worldFor:
Room.combat => _combat,    // add before the closing brace
```

---

7. CombatRoomWorld + FighterComponent (Flame)

lib/game/rooms/combat_room.dart

```
import 'dart:math';
import 'package:flame/components.dart';
import 'package:flame/particles.dart';
import 'package:flutter/material.dart' show Color, Paint;

class CombatRoomWorld extends World {
  late final FighterComponent playerFighter;
  late final FighterComponent enemyFighter;

  @override
  Future<void> onLoad() async {
    await super.onLoad();
    add(RectangleComponent(
      size: Vector2(400, 700), position: Vector2(-200, -350),
      paint: Paint()..color = const Color(0xFF04050A)));
    add(RectangleComponent(
      size: Vector2(360, 3), position: Vector2(-180, 80),
      paint: Paint()..color = const Color(0xFF1A2040)));
    add(RectangleComponent(
      size: Vector2(360, 1), position: Vector2(-180, 82),
      paint: Paint()..color = const Color(0xFF22E1FF)));

    playerFighter = FighterComponent(isPlayer: true, position: Vector2(-90, 60));
    enemyFighter = FighterComponent(isPlayer: false, position: Vector2(90, 60));
    add(playerFighter);
    add(enemyFighter);
  }

  void showHit({required bool attackerIsPlayer}) {
    if (attackerIsPlayer) {
      enemyFighter.flashHit();
      playerFighter.lungeForward();
    } else {
      playerFighter.flashHit();
```

```
      enemyFighter.lungeForward();
    }
  }

  void showOutcome({required bool playerWon}) {
    if (playerWon) { playerFighter.celebrate(); enemyFighter.collapse(); }
    else           { enemyFighter.celebrate(); playerFighter.collapse(); }
  }

  void reset() { playerFighter.reset(); enemyFighter.reset(); }
}

class FighterComponent extends RectangleComponent {
  FighterComponent({required this.isPlayer, required super.position})
      : super(size: Vector2(44, 60), anchor: Anchor.bottomCenter);

  final _rng = Random();
  Vector2? _originPos;
  double _lunge = 0, _hitFlash = 0, _celebrateT = 0;
  bool _collapsed = false;

  static const _lungeDir = 30.0;
  static const _flashDur = 0.18;
  static const _lungeDur = 0.15;

  Color get _baseColor =>
      isPlayer ? const Color(0xFF22E1FF) : const Color(0xFFFF3CAC);

  @override
  void onMount() {
    super.onMount();
    _originPos = position.clone();
    paint.color = _baseColor;
  }

  @override
  void update(double dt) {
    super.update(dt);
    if (_lunge > 0) {
      _lunge = (_lunge - dt / _lungeDur).clamp(0, 1);
      final dir = isPlayer ? 1.0 : -1.0;
      position.x = (_originPos?.x ?? 0) + dir * _lungeDir * _lunge;
    }
    if (_hitFlash > 0) {
      _hitFlash = (_hitFlash - dt / _flashDur).clamp(0, 1);
      paint.color = Color.lerp(_baseColor, const Color(0xFFFFFFFF), _hitFlash)!;
    }
    if (_celebrateT > 0) {
      _celebrateT -= dt;
      position.y = (_originPos?.y ?? 0) + 4 * sin(_celebrateT * 8);
    }
    if (_collapsed && opacity > 0.05)
      opacity = (opacity - dt * 0.6).clamp(0.05, 1.0);
  }

  void lungeForward() => _lunge = 1.0;
  void flashHit() { _hitFlash = 1.0; _emitHitParticles(); }
  void celebrate() => _celebrateT = 2.5;
  void collapse() => _collapsed = true;
  void reset() {
    _collapsed = false; _lunge = 0; _hitFlash = 0; _celebrateT = 0;
    opacity = 1.0; paint.color = _baseColor;
    if (_originPos != null) position.setFrom(_originPos!);
  }

  void _emitHitParticles() {
    final c = _baseColor;
    parent?.add(ParticleSystemComponent(
      position: position - Vector2(0, 20),
```

```
      particle: Particle.generate(
        count: 12, lifespan: 0.45,
        generator: (i) => AcceleratedParticle(
          acceleration: Vector2((_rng.nextDouble()*2-1)*40, -30),
          speed: Vector2((_rng.nextDouble()*2-1)*70, -40-_rng.nextDouble()*40),
          child: CircleParticle(
            radius: 2+_rng.nextDouble()*2.5,
            paint: Paint()..color = c)))));
  }
}
```

---

8. CombatOverlay (Flutter)

lib/ui/overlays/combat_overlay.dart

```
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../data/providers.dart';
import '../../../data/combat/combat_notifier.dart';
import '../../../domain/combat/resolved_stats.dart';
import '../../../game/robot_game.dart';
import '../theme.dart';

class CombatOverlay extends ConsumerStatefulWidget {
  const CombatOverlay({super.key, required this.game});
  final RoboPetGame game;
  @override
  ConsumerState<CombatOverlay> createState() => _CombatOverlayState();
}

class _CombatOverlayState extends ConsumerState<CombatOverlay> {
  static const _turnInterval = Duration(milliseconds: 1600);
  Timer? _timer;

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) => _startBattle());
  }

  void _startBattle() {
    final playerStats = ref.read(resolvedStatsProvider);
    ref.read(combatProvider.notifier)
        .startBattle(player: playerStats, enemy: ResolvedCombatStats.ghost);
    _timer?.cancel();
    _timer = Timer.periodic(_turnInterval, (_) => _tick());
  }

  void _tick() {
    final notifier = ref.read(combatProvider.notifier);
    final hit = notifier.tick();
    if (!hit) return;
    final state = ref.read(combatProvider);
    if (state.phase == CombatPhase.battling) {
      widget.game.combatWorld
          .showHit(attackerIsPlayer: state.turnCount % 2 == 1);
    } else {
      _timer?.cancel();
      widget.game.combatWorld
          .showOutcome(playerWon: state.phase == CombatPhase.victory);
    }
```

```
  }

  @override
  void dispose() { _timer?.cancel(); super.dispose(); }

  void _retreat() {
    _timer?.cancel();
    ref.read(combatProvider.notifier).reset();
    widget.game.combatWorld.reset();
    widget.game.navigateTo(Room.laboratory);
  }

  @override
  Widget build(BuildContext context) {
    final cs = ref.watch(combatProvider);
    final playerStats = ref.watch(resolvedStatsProvider);
    return Stack(children: [
      Positioned(
        top: MediaQuery.of(context).padding.top + 52,
        left: 12, right: 12,
        child: _StatsRow(
          playerHpFrac: cs.playerHpFraction,
          enemyHpFrac: cs.enemyHpFraction,
          playerStats: playerStats,
          enemyStats: ResolvedCombatStats.ghost)),
      Positioned(
        bottom: 88, left: 12, right: 12,
        child: _BattleLog(log: cs.log)),
      if (cs.phase == CombatPhase.victory || cs.phase == CombatPhase.defeat)
        Center(child: _OutcomeBanner(
            won: cs.phase == CombatPhase.victory, onRetreat: _retreat)),
      if (cs.phase == CombatPhase.battling)
        Positioned(
          bottom: 88, right: 12,
          child: TextButton(
            onPressed: _retreat,
            child: const Text('Retreat',
                style: TextStyle(color: Colors.white38, fontSize: 12)))),
    ]);
  }
}

class _StatsRow extends StatelessWidget {
  const _StatsRow({required this.playerHpFrac, required this.enemyHpFrac,
      required this.playerStats, required this.enemyStats});
  final double playerHpFrac, enemyHpFrac;
  final ResolvedCombatStats playerStats, enemyStats;
  @override
  Widget build(BuildContext context) => Row(children: [
    Expanded(child: _FighterCard(label: 'YOUR ROBOT', color: AppColors.cyan,
        hpFrac: playerHpFrac, atk: playerStats.attack, def: playerStats.defense)),
    const SizedBox(width: 10),
    Text('VS', style: TextStyle(
        color: Colors.white38, fontSize: 11, fontWeight: FontWeight.bold)),
    const SizedBox(width: 10),
    Expanded(child: _FighterCard(label: 'GHOST', color: AppColors.magenta,
        hpFrac: enemyHpFrac, atk: enemyStats.attack, def: enemyStats.defense)),
  ]);
}

class _FighterCard extends StatelessWidget {
  const _FighterCard({required this.label, required this.color,
      required this.hpFrac, required this.atk, required this.def});
  final String label; final Color color;
  final double hpFrac; final int atk, def;
  @override
  Widget build(BuildContext context) => Container(
    padding: const EdgeInsets.all(8),
    decoration: BoxDecoration(
```

```
      color: Colors.black54,
      border: Border.all(color: color.withOpacity(0.5)),
      borderRadius: BorderRadius.circular(8)),
    child: Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
      Text(label, style: TextStyle(
          color: color, fontSize: 9, letterSpacing: 1.5, fontWeight: FontWeight.bold)),
      const SizedBox(height: 4),
      Stack(children: [
        Container(height: 6, decoration: BoxDecoration(
            color: Colors.white12, borderRadius: BorderRadius.circular(3))),
        FractionallySizedBox(widthFactor: hpFrac,
          child: Container(height: 6, decoration: BoxDecoration(
              color: color, borderRadius: BorderRadius.circular(3),
              boxShadow: [BoxShadow(color: color.withOpacity(0.5), blurRadius: 4)]))),
      ]),
      const SizedBox(height: 4),
      Row(mainAxisAlignment: MainAxisAlignment.spaceBetween, children: [
        Text('ATK $atk', style: const TextStyle(color: Colors.white54, fontSize: 9)),
        Text('DEF $def', style: const TextStyle(color: Colors.white54, fontSize: 9)),
      ]),
    ]));
}

class _BattleLog extends StatelessWidget {
  const _BattleLog({required this.log});
  final List<String> log;
  @override
  Widget build(BuildContext context) {
    final recent = log.length > 5 ? log.sublist(log.length - 5) : log;
    return Container(
      padding: const EdgeInsets.all(10),
      decoration: BoxDecoration(
          color: Colors.black87,
          border: Border.all(color: Colors.white12),
          borderRadius: BorderRadius.circular(8)),
      child: Column(crossAxisAlignment: CrossAxisAlignment.start,
          children: recent.reversed.map((line) => Padding(
            padding: const EdgeInsets.only(bottom: 2),
            child: Text(line, style: const TextStyle(
                color: Colors.white70, fontSize: 10)))).toList()));
  }
}

class _OutcomeBanner extends StatelessWidget {
  const _OutcomeBanner({required this.won, required this.onRetreat});
  final bool won; final VoidCallback onRetreat;
  @override
  Widget build(BuildContext context) {
    final c = won ? AppColors.green : Colors.redAccent;
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 32, vertical: 20),
      decoration: BoxDecoration(
          color: Colors.black87, border: Border.all(color: c, width: 2),
          borderRadius: BorderRadius.circular(16),
          boxShadow: [BoxShadow(color: c.withOpacity(0.35), blurRadius: 24)]),
      child: Column(mainAxisSize: MainAxisSize.min, children: [
        Text(won ? 'VICTORY' : 'DEFEATED',
            style: TextStyle(color: c, fontSize: 28,
                fontWeight: FontWeight.bold, letterSpacing: 3)),
        const SizedBox(height: 16),
        GestureDetector(
          onTap: onRetreat,
          child: Container(
            padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 10),
            decoration: BoxDecoration(
                border: Border.all(color: c), borderRadius: BorderRadius.circular(8)),
            child: Text('Return to Lab',
                style: TextStyle(color: c, fontWeight: FontWeight.bold)))),
      ]));
```

```
  }
}
```

---

9. LoadoutOverlay: Deploy to Battle button (diff)

Add these two items to the Column in LoadoutOverlay.build, after the SizedBox slot row. Nothing else changes.

```
// After the slot row SizedBox:
const SizedBox(height: 10),
Padding(
  padding: const EdgeInsets.symmetric(horizontal: 16),
  child: _DeployButton(equipped: equipped, game: game, ref: ref)),

// ---- New widget at bottom of loadout_overlay.dart: ----
class _DeployButton extends StatelessWidget {
  const _DeployButton({
      required this.equipped, required this.game, required this.ref});
  final Map<ModuleSlot, ModuleInstance> equipped;
  final RoboPetGame game;
  final WidgetRef ref;

  @override
  Widget build(BuildContext context) {
    final ready = equipped.isNotEmpty;
    return GestureDetector(
      onTap: ready ? () async {
        final stats = ref.read(resolvedStatsProvider);
        final syncService = ref.read(combatSyncProvider);
        unawaited(syncService.push(
          robotInstanceId: 'robot_0',
          stats: stats,
          equipped: equipped.values.toList()));
        game.navigateTo(Room.combat);
      } : null,
      child: AnimatedContainer(
        duration: const Duration(milliseconds: 200),
        padding: const EdgeInsets.symmetric(vertical: 13),
        alignment: Alignment.center,
        decoration: BoxDecoration(
          color: ready ? AppColors.magenta.withOpacity(0.15) : Colors.white10,
          border: Border.all(color: ready ? AppColors.magenta : Colors.white24),
          borderRadius: BorderRadius.circular(10),
          boxShadow: ready ? [BoxShadow(color: AppColors.magenta.withOpacity(0.3), blurRadius:
14)] : []),
        child: Row(mainAxisAlignment: MainAxisAlignment.center, children: [
          Icon(Icons.sports_kabaddi,
              color: ready ? AppColors.magenta : Colors.white30, size: 18),
          const SizedBox(width: 8),
          Text(
            ready ? 'Deploy to Battle (${equipped.length} modules)'
                  : 'Equip at least 1 module to deploy',
            style: TextStyle(
              color: ready ? AppColors.magenta : Colors.white30,
              fontWeight: FontWeight.bold, fontSize: 13)),
        ])));
  }
}
```

---

10. Register overlay in GameScreen

Add one import and one entry to overlayBuilderMap in game_screen.dart. Nothing else changes.

```
import 'overlays/combat_overlay.dart';

// Inside overlayBuilderMap:
kOverlayCombat: (ctx, game) =>
    CombatOverlay(game: game as RoboPetGame),
```

---

11. Verification checklist

- [ ] dart run build_runner build succeeds; combat_snapshot.g.dart generated.
- [ ] resolveStats([]) returns HP:100 ATK:15 DEF:10; equipping Mecanum + Jetson Nano raises ATK and SPD in resolvedStatsProvider.
- [ ] Laboratory Room: Deploy button is dimmed with no modules; glowing magenta with >= 1 equipped.
- [ ] Tapping Deploy navigates to Combat Room; stats row shows your resolved stats vs the ghost.
- [ ] Every 1.6 s: attacker lunges, defender flashes white with particle hits; HP bars drain.
- [ ] When HP hits 0: Victory or Defeated banner appears with correct glow color.
- [ ] Return to Lab resets both the Flame world and CombatNotifier cleanly; second fight works.
- [ ] Firestore console shows users/{uid}/combatSnapshot/active populated after Deploy.

Phase 3 complete. Next: Phase 4 -- Extreme Sudoku Minigame (logic engine, difficulty scaling, claimMinigameReward Cloud Function, rareCogs payout, dedicated Sudoku Room).